## Accessing Files

Command line arguments and environment variables are fine for passing small amounts of data into your program, but for larger amounts of data you are going to need to use something more robust, such as files. It's a little early in the book to get deep into what files really are: Chapter 8, "Strings," is where we'll look at what text data really is. However, this is a good point to look at how to read and write data to the filesystem. Additionally, it's important to realize that the filesystem of *nix-based system is also used to abstract network connections and data streams in general. Because of this, the PHP functions that interact with files can also be used to stream data from network connections (which we'll get to shortly), but first let's look at the basics. For this example, we have a simple text file on the path `/data/sample.txt`, the contents of which are designed to provoke meaningful conversations, and we're going to get PHP to create a string variable from the contents:

```
$fullText = file_get_contents(__DIR__ . '/data/sample.txt');
print $fullText;
Output:
Star Trek: Enterprise was underrated.
The Xindi were better antagonists than the Suliban.
```

Controversial, possibly, but it serves its purpose. This example uses features that are fully explained in Chapter 3, "Functions" and Chapter 8, "Strings." If this is the first programming language you've encountered, then feel free to take a look at those chapters to get more comfortable with those concepts before returning here. As an alternative to `file_get_contents()`, the `file()` function will create an array where each element is one line of the file:

```
$lines = file(__DIR__ . '/data/sample.txt');
print_r($lines);

Output:
Array
(
    [0] => Star Trek: Enterprise was underrated.
    [1] => The Xindi were better antagonists than the Suliban.
)
```